

Data Types:

The first type of data structure i.e. **primitive data structure** is considered as the fundamental data structure and it allows storing the values of only one data type but **non-primitive data** structures are considered as the user-defined structure that allows storing values of different data types within one entity. The primitive data structure consists of fundamental data types like float, character, integer, etc.

Primitive Data Structures:

These are the most primitive or basic data structures. They are the building blocks for data manipulation and contain pure, simple values of data. Python has four primitive variable types:

- Integers
- Float
- Strings
- Boolean

In the next sections, you'll learn more about them!

Integers:

You can use an integer to represent numeric data and, more specifically, whole numbers from negative infinity to infinity, like 4, 5, or -1.

Float:

"Float" stands for 'floating point number'. You can use it for rational numbers, usually ending with a decimal figure, such as 1.11 or 3.14.

Take a look at the following DataCamp Light Chunk and try out some of the integer and float operations!

String:

Strings are collections of alphabets, words, or other characters. In Python, you can create strings by enclosing a sequence of characters within a pair of single or double quotes. For example: 'cake', "cookie", etc.

```
x = 'Cake'  
y = 'Cookie'
```

Boolean:

This built-in data type can take up the values: True and False, which often makes them interchangeable with the integers 1 and 0. Booleans are useful in conditional and comparison expressions, just like in the following examples:

```
x = 4  
y = 2  
x == y
```

Output:

False

To check the type of an object in Python, use the built-in type() function, just like in the lines of code below:

```
i = 4.0  
type(i)
```

Output:

Float

Non-Primitive Data Structures:

Non-primitive data structures are the data structure created by the programmer with the help of primitive data structures. Non-primitive

data structures are divided into **linear and non-linear data structures**. lists, tuples, dict and sets are examples of linear data structure and trees, graph are examples of non-linear data structure.

List:

Lists in Python are used to store collections of heterogeneous items. These are mutable, which means that you can change their content without changing their identity. You can recognize lists by their square brackets [and] that hold elements separated by a comma ,.

The list has the following features -

- You can use Python lists to store data of multiple types simultaneously.
- Lists help preserve data sequences and further process those sequences in other ways.
- Lists are dynamic.
- Lists are mutable.
- Lists are ordered.
- An index is used to traverse a list.

```

In [3]: x = []                # Empty list.
        print(type(x))

        x1 = [1,2,3]          #Initiated list with elements.
        print(type(x1))

        x2 = list([1,'apple',3]) #another type of initiated the list.
        print(type(x2))

        x2[1] = 'orange'      #replacing the element of 1st index position of list x2.
        print(x2)

<class 'list'>
<class 'list'>
<class 'list'>
[1, 'orange', 3]

```

Some of the in-built methods of list and their explanations:

```

In [ ]:

In [4]: list_num = [1,2,45,6,7,2,90,23,435]
        list_char = ['c','o','o','k','i','e']

        list_num.append(11) # Add 11 to the list, by default adds to the last position
        print(list_num)

[1, 2, 45, 6, 7, 2, 90, 23, 435, 11]

```

```
In [5]: list_num.insert(0, 11) #inserting elements in some particular index position in this case
        #zero index position.
        print(list_num)
        [11, 1, 2, 45, 6, 7, 2, 90, 23, 435, 11]
```

```
In [6]: list_char = ['c','o','o','k','i','e']
        list_char.remove('o') #Remove the first occurrence of 'o' from list_char with the
                               #help of remove()
        print(list_char)
        ['c', 'o', 'k', 'i', 'e']
```

```
In [ ]:
```

```
In [7]: list_char.pop(-2) # Removes the item at the specified position
        print(list_char)
        ['c', 'o', 'k', 'e']
```

```
In [8]: list_num.sort() # In-place sorting
        print(list_num)
        [1, 2, 2, 6, 7, 11, 11, 23, 45, 90, 435]
```

```
In [ ]:
```

```
In [ ]:
```

```
In [9]: list.reverse(list_num)
        print(list_num)
        [435, 90, 45, 23, 11, 11, 7, 6, 2, 2, 1]
```

```
In [ ]:
```

```
In [10]: del list_num #keyword to delete the existence of the list
```

```
In [ ]:
```

Tuples

Tuples are another standard sequence data type. The difference between tuples and list is that tuples are immutable, which means once defined you cannot delete, add or edit any values inside it. This might be useful in situations where you might want to pass the control to someone else but you do not want them to manipulate data in your collection, but rather

maybe just see them or perform operations separately in a copy of the data.

A python tuple has the following features -

- Tuples are used to store heterogeneous and homogeneous data.
- Tuples are immutable in nature.
- Tuples are ordered
- An index is used to traverse a tuple.
- Tuples are similar to lists. It also preserves the data sequence.

As tuples are immutable, they are faster than the list because they are static.

```
In [11]: x_tuple = 1,2,3,4,5
         y_tuple = ('c','a','k','e')
         x_tuple[0]

Out[11]: 1

In [ ]:

In [12]: y_tuple[3]
         x_tuple[0] = 0 # Cannot change values inside a tuple

-----
TypeError                                 Traceback (most recent call last)
Input In [12], in <cell line: 2>()
      1 y_tuple[3]
----> 2 x_tuple[0] = 0

TypeError: 'tuple' object does not support item assignment
```

Difference between Lists and Tuples:

	Python Lists	Python Tuples
1	List are mutable	Tuples are immutable

2	Iterations are time-consuming	Iterations are comparatively Faster
3	Inserting and deleting items is easier with a list.	Accessing the elements is best accomplished with a tuple data type.
4	Lists consume more memory	Tuple consumes less than the list
5	Lists have several built-in methods.	A tuple does not have many built-in methods because of immutability
6	A unexpected change or error is more likely to occur in a list.	In a tuple, changes and errors don't usually occur because of immutability.